

Registros de Decisão Arquitetural (ADRs)

SynapseEdu — Hub Inteligente de Recursos Educacionais

ADR 01: Escolha do Framework Backend

Status: Aprovado

Contexto: Precisamos de uma API RESTful robusta, com validação de dados nativa e tipagem forte em Python para mitigar erros em produção.

Decisão: Escolhemos o FastAPI.

Consequências:

- Positivas: alta performance, documentação automática com Swagger integrada e uso nativo do Pydantic para validações de payloads da API.
- Negativas: menos opinativo que o Django, exigindo que a estrutura de pastas e as conexões do ORM sejam criadas manualmente (o que é positivo para o aprendizado técnico).

ADR 02: Padrão de Separação de Camadas (Models vs Schemas)

Status: Aprovado

Contexto: Ao lidar com dados vindos de requisições HTTP e dados que vão para o banco PostgreSQL, misturar as responsabilidades pode causar falhas de segurança ou envio de dados desnecessários.

Decisão: Separar estritamente as classes em models.py (SQLAlchemy) e schemas.py (Pydantic).

Consequências:

- Positivas: segurança e isolamento. É possível criar um schema de entrada que impede o cliente de injetar um id arbitrário, mantendo validação estrita no model.
- Negativas: leve duplicação na declaração de campos em ambos os arquivos, justificada pelo ganho arquitetural.

ADR 03: Estratégia de Chave Primária (UUID vs Integer)

Status: Aprovado

Contexto: A tabela resources precisa de uma chave primária. As duas opções candidatas eram Integer autoincremento e UUID.

Decisão: Utilizar UUID como chave primária de resources.

Consequências:

- Positivas: não expõe o volume total de registros do sistema (diferente de um id sequencial como /resources/5), não é previsível/adivinhável por terceiros e é o padrão adotado em APIs públicas modernas — reforça a percepção de maturidade técnica do projeto.
- Negativas: índice ligeiramente mais pesado que um Integer e chaves menos legíveis em debugging manual; ambos aceitáveis dado o volume de dados esperado neste projeto.

ADR 04: Estrutura de Armazenamento de Tags (Array vs Tabela de Relacionamento)

Status: Aprovado

Contexto: Cada recurso deve armazenar até 3 tags recomendadas, geradas manualmente ou pela IA (Smart Assist). Era necessário decidir entre um campo ARRAY nativo do PostgreSQL ou uma tabela de relacionamento N:N (resources ↔ tags).

Decisão: Utilizar um campo tags do tipo ARRAY(VARCHAR) diretamente na tabela resources.

Consequências:

- Positivas: implementação mais simples e direta, suficiente para o escopo atual (não há requisito de reaproveitamento de tags entre recursos nem tela de gestão de tags).
- Negativas: não impede duplicidade/variação de escrita entre tags (ex.: "Matemática" vs "matemática") e dificulta consultas agregadas por tag (ex.: "quantos recursos usam a tag X"). Caso o projeto evolua para exigir isso, a migração para uma tabela de relacionamento deve ser revisitada (ver Modelo de Dados, seção de evolução futura).

ADR 05: Ambiente do Banco de Dados no Desenvolvimento (Local vs Nuvem)

Status: Aprovado

Contexto: Era necessário decidir se o PostgreSQL usado durante o desenvolvimento seria instalado localmente na máquina do desenvolvedor ou provisionado em um serviço de nuvem com free tier (ex.: Neon, Supabase) desde o início do projeto.

Decisão: Utilizar PostgreSQL local durante a fase de desenvolvimento e validação inicial da aplicação (models, CRUD, integração com a IA). A migração para um banco em nuvem (Neon) foi efetivada após a estabilização do CRUD e das migrations via Alembic, trocando apenas o valor de DATABASE_URL no .env.

Consequências:

- Positivas: reduz variáveis externas (rede, autenticação de terceiros, limites de free tier) enquanto a integração entre SQLAlchemy, Pydantic e FastAPI ainda está sendo validada pela primeira vez; erros de conexão ficam mais fáceis de diagnosticar por serem 100% locais.
- Positivas: a migração para nuvem exige apenas a troca do valor de DATABASE_URL no .env, sem alteração de código — consequência direta do ADR-02 (config centralizada) e do uso de pydantic-settings, o que valida na prática o valor dessas decisões anteriores.
- Negativas: o ambiente de desenvolvimento local não replica 100% as condições de produção (latência de rede, comportamento sob múltiplas conexões remotas), portanto a migração para nuvem antes da entrega final é necessária para validar o cenário mais próximo do real.
- Alternativa considerada e rejeitada (containerização): cogitou-se fazer o docker-compose.yml conectar diretamente ao PostgreSQL do Neon, sem um serviço de banco local, para que qualquer pessoa executando o ambiente via Docker visualizasse o mesmo acervo de dados. A alternativa foi descartada: exigiria distribuir a DATABASE_URL real de produção para qualquer pessoa que rodasse o Docker, permitindo que essa pessoa escrevesse/apagasse dados no banco real do projeto. O docker-compose.yml da entrega final mantém um serviço PostgreSQL local e isolado, consistente com o restante deste ADR.

ADR 06: Provedor de LLM para o Smart Assist (Gemini vs Groq)

Status: Aprovado

Contexto: O desafio técnico permite o uso de OpenAI, Gemini, Anthropic ou Groq como provedor de LLM. A integração inicial foi implementada com o Google Gemini API, porém o free tier exigiu vinculação de cartão de

crédito para funcionamento estável, o que inviabilizou a validação contínua da funcionalidade Smart Assist durante o desenvolvimento.

Decisão: Adotar a Groq API como provedor principal do Smart Assist (modelo llama-3.3-70b-versatile), mantendo a integração com o Gemini como opcional (chave não obrigatória em core/config.py). A camada de serviço (ai_service.py) permanece isolada do restante da aplicação, conforme ADR de separação de camadas do Nível 3 do Modelo C4.

Consequências:

- Positivas: chave de API gratuita obtida sem cartão de crédito, com limites de requisição generosos para o contexto de desenvolvimento e demonstração do desafio; suporte nativo a resposta estruturada em JSON (response_format={"type": "json_object"}), equivalente ao response_mime_type usado na integração original com o Gemini.
- Positivas: a troca de provedor exigiu alteração isolada em app/services/ai_service.py e app/core/config.py, sem qualquer alteração em routers, schemas ou crud — validação prática, na segunda vez, do valor da arquitetura de camadas definida desde o início do projeto.
- Negativas: o modelo llama-3.3-70b-versatile (Groq) é um modelo de propósito geral open-source, e não um modelo proprietário otimizado como o Gemini; a qualidade das descrições geradas deve ser validada empiricamente durante o desenvolvimento do Smart Assist.
- Negativas: a documentação de Engenharia de Prompt precisou ser atualizada para refletir o novo provedor (ver documento correspondente).